

DD2350 Algoritmer, datastrukturer och komplexitet

Mästarprov 1

Lösningarna, uppgift för uppgift – och felen som återkom

KTH · HT 2026

I dag

1. **Sum Ergo Cogito** – dekomposition när multiplikation kostar.
2. **Hem för hemlösa katter** – tvåfärgning och en delmängdssumma med små tal.
3. **Fikaoptimering** – dynamisk programmering under ett tak på 42.
4. **Formalia** – det som kostade utan att ha med algoritmerna att göra.

Varje uppgift i samma ordning

Uppgiften, idén, pseudokoden, tidskomplexiteten och korrektheten. Sist de vanligaste felen.

Lösningsförslaget finns i Canvas. Det här är samma lösningar, med bilder, och med felen som återkom i inlämningarna.

Del 1 av 3

Sum Ergo Cogito

dekomposition när multiplikation kostar



Uppgiften och kostnadsmodellen

Beräkna **siffersumman** $s(x)$ av ett heltal x med n siffror:

$$s(2026) = 2 + 0 + 2 + 6 = 10.$$

- Konstruktionsmetod: **dekomposition**.
- **Snabbare än kvadratisk** i n.
- Pseudokod, tidskomplexitet och en beskrivning i ord och **bild**.
- Inget korrekthetsbevis, men en redogörelse för **vad ett bevis skulle visa**.

Kostnadsmodellen

addition, subtraktion, jämförelse	$O(n)$
multiplikation, division, modulo	$O(n^{1,5})$
ett konstant tal	$O(1)$
$digs(x)$, antalet siffror i x	$O(1)$

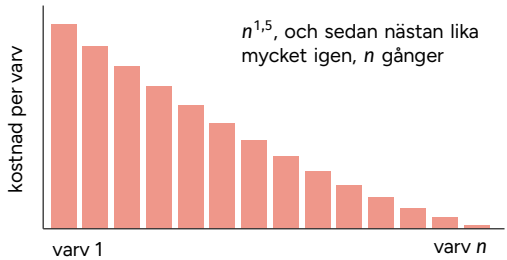
n är antalet siffror i den största operanden.

Det finns ingen sifferlista

Inget får antas om hur talen lagras. Varje siffra måste räknas fram med div och mod, och de kostar efter talens storlek.

Den självklara slingan är för långsam

```
SiffersummaNaiv(x) =
1    s ← 0
2    while x > 0 do
3        s ← s + (x mod 10)
4        x ← x div 10
5    return s
```



n varv, och varv k gör div och mod på ett tal med $n - k + 1$ siffror:

$$\sum_{k=1}^{n} O\big((n-k+1)^{1,5}\big) = O\big(n^{2,5}\big).$$

Modellen ger bara övre gränser, och bättre än så går inte att visa. $n^{2,5}$ är inte under n^2.

En siffra i taget är för lite

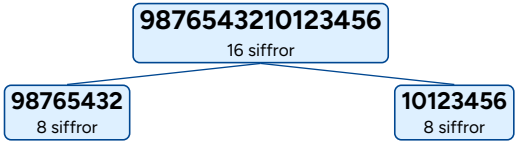
Att skala av en siffra rekursivt är också dekomposition, men $T(n) = T(n-1) + O(n^{1,5})$ ger bara $O(n^{2,5})$. Delarna måste vara ungefär lika stora.

Idén: dela talet på mitten

9876543210123456
16 siffror

$x = a \cdot 10^m + b$
med $m = \lfloor n/2 \rfloor = 8$

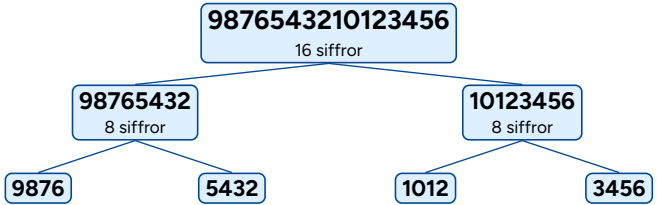
Idén: dela talet på mitten



$x = a \cdot 10^m + b$
med $m = \lfloor n/2 \rfloor = 8$

$s(x) = s(a) + s(b)$:
två hälften så stora problem

Idén: dela talet på mitten

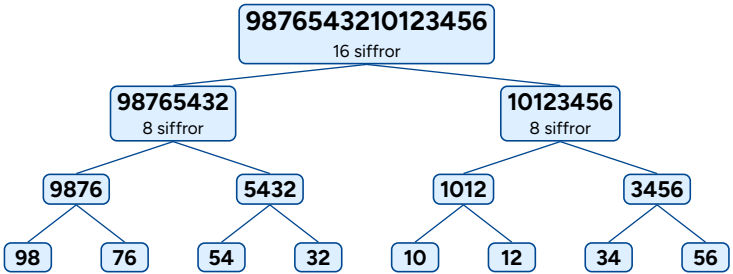


$x = a \cdot 10^m + b$
med $m = \lfloor n/2 \rfloor = 8$

$s(x) = s(a) + s(b)$:
två hälften så stora problem

$98765432 = 9876 \cdot 10^4 + 5432$

Idén: dela talet på mitten



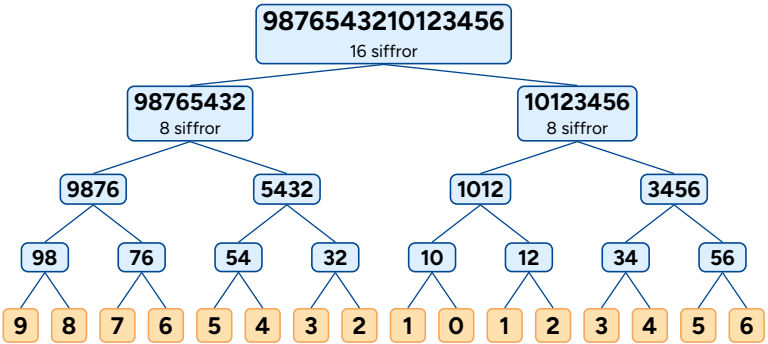
$x = a \cdot 10^m + b$
med $m = \lfloor n/2 \rfloor = 8$

$s(x) = s(a) + s(b)$:
två hälften så stora problem

$98765432 = 9876 \cdot 10^4 + 5432$

$9876 = 98 \cdot 10^2 + 76$

Idén: dela talet på mitten



$s(9876543210123456) = s(98765432) + s(10123456) = 44 + 22 = \mathbf{66}$

$x = a \cdot 10^m + b$
med $m = \lfloor n/2 \rfloor = 8$

$s(x) = s(a) + s(b)$:
två hälften så stora problem

$98765432 = 9876 \cdot 10^4 + 5432$

$9876 = 98 \cdot 10^2 + 76$

basfall, en siffra: $s(d) = d$

Bilden i lösningsförslaget anger summan 76. Räkna efter: det blir 66.

Pseudokod

```
   SumErgoCogito(x) =
1      if x < 0 then x ← 0 − x
2      return SumDigits(x)

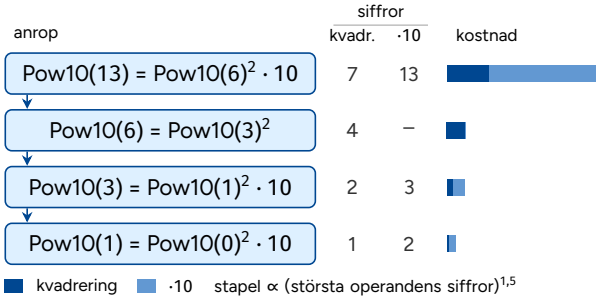
   SumDigits(x) =
       // PRE: x ≥ 0. POST: returnerar s(x)
3      d ← digs(x)
4      if d ≤ 1 then return x
5      m ← d div 2
6      q ← Pow10(m)
7      a ← x div q
8      b ← x mod q
9      return SumDigits(a) + SumDigits(b)
```

Tecknet först: div avrundar nedåt, så −305 **div** 10 = −31. Utan rad 1 blir s(−305) 10 i stället för 8.

```
   Pow10(m) =
       // PRE: m ≥ 0. POST: returnerar 10^m
10     if m = 0 then return 1
11     h ← Pow10(m div 2)
12     r ← h mul h
13     if m mod 2 = 1 then r ← r mul 10
14     return r
```

- **Delningen:** a har $\lceil n/2 \rceil$ siffror, b högst $\lfloor n/2 \rfloor$. Inledande nollor i b bidrar ändå med 0.
- **Basfallet:** $digs(0) = 1$, så $b = 0$ ger 0.
- **Lösningsförslaget** förberäknar 10^{2^i} och multiplicerar ihop 10^m av dem (Initialize, TenExp).

10^m: kvadrera, multiplicera inte med tio i taget



10^m är inget konstant tal

m växer med n. Bara tal som är bestämda i förväg, som 10, kostar $O(1)$.

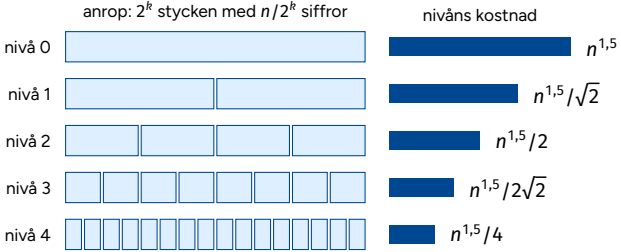
Kvadrering: $O(m^{1,5})$

Översta nivån kvadrerar $\lfloor m/2 \rfloor + 1$ siffror och tar kanske m siffror gånger 10: båda $O(m^{1,5})$. Nedåt halveras storleken: en geometrisk summa.

Tio i taget: bara $O(m^{2,5})$

Den k:te multiplikationen med 10 har en operand med k siffror, och $\sum_k O(k^{1,5}) = O(m^{2,5})$.

Tidskomplexitet: översta nivån dominerar



Nivå k kostar $2^k\left(n/2^k\right)^{1,5} = n^{1,5}\,2^{-k/2}$,
en avtagande geometrisk serie, så
$T(n) \le c\,n^{1,5}\left(1 + 2^{-1/2} + 2^{-1} + \ldots\right) \approx 3{,}41\,c\,n^{1,5}$.

Mästarsatsen

$T(n) = 2\,T(n/2) + O(n^{1,5})$

$a = 2,\ b = 2,\ n^{\log_b a} = n.$

$n^{1,5}$ växer polynomiskt snabbare, så $T(n) = O(n^{1,5})$.

- Med lösningsförslagets TenExp, $O(m^{1,5} \log m)$, blir det $O(n^{1,5} \log n)$. Båda är snabbare än kvadratiskt.
- Additionen $s(a) + s(b)$ kostar bara $O(\log n)$: delsummorna är högst $9n$.
- Skriv O, inte Θ: modellen ger bara övre gränser.

Korrekthet: vad ett bevis skulle visa

1. **Specifikationen:** för varje heltal x stannar algoritmen och returnerar $s(x)$, siffersumman av $|x|$.
2. **Terminering:** antalet siffror minskar strikt i varje anrop, eftersom $\lceil n/2 \rceil < n$ och $\lfloor n/2 \rfloor < n$ när $n \geq 2$.
3. **Rätt svar**, med induktion över n. Basfall: $s(x) = x$ när $n = 1$. Steg: om $x = a \cdot 10^m + b$ med $0 \leq b < 10^m$ är siffrorna i x siffrorna i a följda av b skrivet med m siffror. Alltså $s(x) = s(a) + s(b)$.
4. **Hjälpfunktionerna** uppfyller sina in- och utgångsvillkor: Pow10(m) returnerar 10^m, och varje anrop uppfyller förvillkoren.

Kravet för E

Inget bevis, bara principerna: vad som ska visas, och med vilka verktyg.

Det räcker inte med

”Den ger rätt svar på mina testfall.” Test kan visa att något är fel, aldrig att allt är rätt. Inte heller bara terminering, eller bara rätt svar.

Vanliga fel i uppgift 1

- 10^m **som konstant.** Bara tal som är bestämda i förväg kostar $O(1)$, och m växer med n.
- **Tio i taget.** För m multiplikationer med 10 går bara $O(m^{2,5})$ att visa, och då blir hela algoritmen $O(n^{2,5})$.
- **Enhetskostnad.** ”Pow10 gör log m multiplikationer, alltså $O(\log m)$.” Varje multiplikation kostar efter operandernas storlek.
- **En siffra i taget**, i en slinga eller rekursivt: $O(n^{2,5})$, inte under n^2.
- **Ingen tight analys.** ”log n nivåer à $n^{1,5}$” för en algoritm som är $O(n^{1,5})$. Nivåerna blir billigare nedåt.
- **Negativa tal.** När div avrundar nedåt blir $s(-305) = 10$. Ett mindre fel, om det inte rättas på redovisningen.
- **Ingen bild.** Den fick ritas på redovisningen.
- **Korrekthet som testning**, eller utan terminering.

Del 2 av 3

Hem för hemlösa katter

tvåfärgning och delmängdssumma



Uppgiften

n hemlösa katter ska placeras i **två rum**.

1. Rum *j* rymmer högst c_j katter.
2. Katt *i* trivs bara i rummen i listan $R_i \subseteq \{1, 2\}$.
3. Paren i listan *B* bråkar om de hamnar i samma rum.

Avgör om alla katter kan placeras utan att något rum förstörs. Ja eller nej räcker, ingen placering behövs.

Polynomisk tid med enhetskostnad räcker. Kursens datastrukturer får användas med sina analyser.

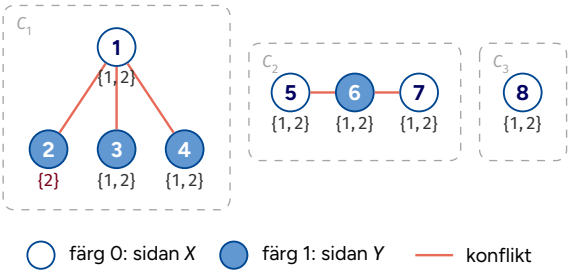
Tre villkor, tre verktyg

3. konflikter → grafen ska vara bipartit: BFS i varje komponent

2. trivsel → förbjuder placeringar av en komponent

1. kapacitet → dynamisk programmering över antalet katter i rum 1

Modellen: konfliktgrafen

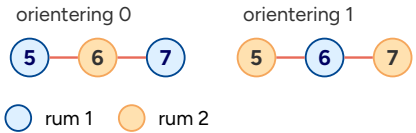


Under varje katt: R_i, rummen den trivs i.

- **Katt = nod, konflikt = kant.** En placering är en tvåfärgning: varje kant måste gå mellan rummen.
- **BFS** från en ofärgad katt färgar dess komponent nivå för nivå: jämna nivåer färg 0, udda färg 1.
- **Exemplet:** $n = 8$, $c_1 = 3$ och $c_2 = 5$. Katt 2 trivs bara i rum 2.

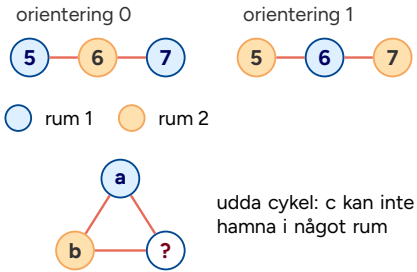
Lösningsförslaget kallar sidorna *A* och *B*. Här heter de *X* och *Y*, eftersom *B* redan är konfliktlistan.

En komponent har högst två placeringar



1. Välj rum för en katt *r*. Varje granne tvingas till det andra rummet, deras grannar tillbaka, och så vidare.
2. Komponenten är sammanhängande, så alla nås: avståndet från *r* modulo 2 bestämmer rummet.
3. Alltså finns högst två placeringar: BFS-färgningen och dess spegelbild.

En komponent har högst två placeringar



Men t komponenter ger 2^t kombinationer

För många att pröva. Komponenterna hänger ihop bara via antalet katter per rum.

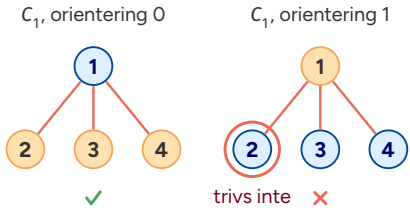
1. Välj rum för en katt r. Varje granne tvingas till det andra rummet, deras grannar tillbaka, och så vidare.
2. Komponenten är sammanhängande, så alla nås: avståndet från r modulo 2 bestämmer rummet.
3. Alltså finns högst två placeringar: BFS-färgningen och dess spegelbild.
4. Ser BFS en kant mellan två katter med samma färg finns en udda cykel. Då finns ingen placering alls, och svaret är nej.

Trivseln stryker orienteringar

Orientering 0: $X \to$ rum 1 och $Y \to$ rum 2.
Tillåten om alla i X trivs i rum 1 och alla i Y i rum 2.

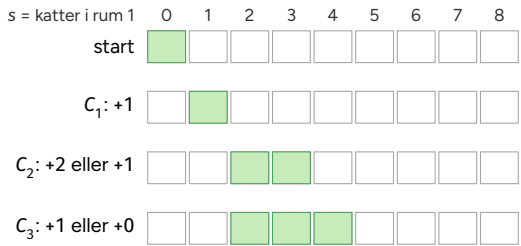
Orientering 1: tvärtom.

	X	Y	o0: i rum 1	o1: i rum 1
C_1	$\{1\}$	$\{2,3,4\}$	1 ✓	3 ✗ (katt 2)
C_2	$\{5,7\}$	$\{6\}$	2 ✓	1 ✓
C_3	$\{8\}$	$\emptyset$	1 ✓	0 ✓



- En katt med $R_i = \emptyset$ stryker båda orienteringarna: svara nej.
- $O(1)$ per katt, eftersom $|R_i| \leq 2$.

Kapaciteten: en delmängdssumma med små tal



$M_i[s]$ är sant om komponent 1..i kan orienteras tillåtet med **exakt** s **katter i rum 1**.

$$M_0[s] = (s = 0)$$

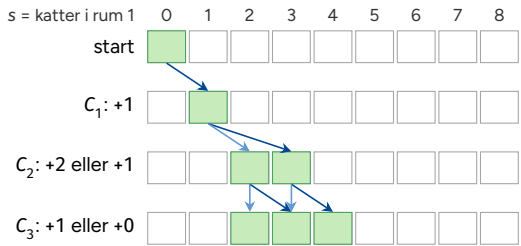
$$M_i[s] = \left(ok_0 \wedge M_{i-1}[s - |X_i|]\right) \vee \left(ok_1 \wedge M_{i-1}[s - |Y_i|]\right)$$

Svaret är ja om $M_t[s]$ för något s med $n - c_2 \le s \le c_1$.

Små tal

s räknar katter, så tabellen har $n + 1$ kolumner, hur stora c_1 och c_2 än är.

Kapaciteten: en delmängdssumma med små tal



$M_i[s]$ är sant om komponent 1..i kan orienteras tillåtet med **exakt** s **katter i rum 1**.

$$M_0[s] = (s = 0)$$

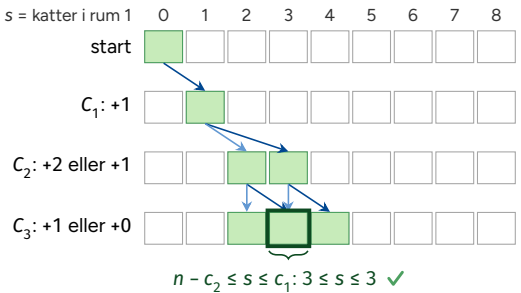
$$M_i[s] = \left(ok_0 \wedge M_{i-1}[s - |X_i|]\right) \vee \left(ok_1 \wedge M_{i-1}[s - |Y_i|]\right)$$

Svaret är ja om $M_t[s]$ för något s med $n - c_2 \le s \le c_1$.

Små tal

s räknar katter, så tabellen har $n + 1$ kolumner, hur stora c_1 och c_2 än är.

Kapaciteten: en delmängdssumma med små tal



$M_i[s]$ är sant om komponent 1..i kan orienteras tillåtet med **exakt** s **katter i rum 1**.

$$M_0[s] = (s = 0)$$

$$M_i[s] = \left(ok_0 \wedge M_{i-1}[s - |X_i|]\right) \vee \left(ok_1 \wedge M_{i-1}[s - |Y_i|]\right)$$

Svaret är ja om $M_t[s]$ för något s med $n - c_2 \le s \le c_1$.

Små tal

s räknar katter, så tabellen har $n + 1$ kolumner, hur stora c_1 och c_2 än är.

Pseudokod

```
    KattHem(n, B, c_1, c_2, R) =
1       bygg grannlistor för grafen G med kanterna B
2       S_1 ← {i : 1 ∈ R_i};  S_2 ← {i : 2 ∈ R_i}
3       färg[1..n] ← −1;  komps ← tom lista
4       for r ← 1 to n do
5           if färg[r] = −1 then
6               BFS från r: färg[v] ← avståndet mod 2,
                samla X (färg 0) och Y (färg 1)
7               if BFS ser en kant med lika färg then
                    return false
8               ok_0 ← X ⊆ S_1 ∧ Y ⊆ S_2
9               ok_1 ← Y ⊆ S_1 ∧ X ⊆ S_2
10              lägg (|X|, |Y|, ok_0, ok_1) sist i komps
11      return Balansera(komps, n, c_1, c_2)
```

```
    Balansera(komps, n, c_1, c_2) =
12      M[0..n] ← false;  M[0] ← true
13      for each (x, y, ok_0, ok_1) ∈ komps do
14          inv M[s] ⇔ de hittills behandlade
                komponenterna kan ge s katter i rum 1
15          M′[0..n] ← false
16          for s ← 0 to n do
17              if M[s] ∧ ok_0 then M′[s + x] ← true
18              if M[s] ∧ ok_1 then M′[s + y] ← true
19          M ← M′
20      for s ← max(0, n − c_2) to min(c_1, n) do
21          if M[s] then return true
22      return false
```

$s + x \le n$ på rad 17–18: $M[s]$ är bara sant för s upp till antalet behandlade katter. Lösningsförslaget löser samma delproblem rekursivt med memoisering: Sök(i, x, y), med kvarvarande kapacitet x och y.

Tidskomplexitet

- **Grannlistor:** $O(n + |B|)$.
- **BFS över alla komponenter:** $O(n + |B|)$. Varje katt köas en gång och varje kant ses två gånger.
- S_1, S_2, ok_0, ok_1: $O(n)$, eftersom $|R_i| \le 2$.
- **Tabellen:** högst n komponenter, $n + 1$ värden och $O(1)$ per värde, alltså $O(n^2)$.
- **Totalt** $O(n^2 + |B|) = O(n^2)$, eftersom $|B| \le n(n-1)/2$ när inget par upprepas.

Indexera inte med kapaciteten

En tabell med $c_1 + 1$ kolumner kostar $O(n \cdot c_1)$. Det är pseudopolynomiskt: c_1 kan vara exponentiellt stort i förhållande till sin längd. Kapa c_j till n.

Lösningsförslaget räknar slappt

Det anger $O(n^4)$, och polynomiskt räcker. Bättre än kvadratiskt går inte i allmänhet: B kan ha $\Theta(n^2)$ par.

Korrekthet

Vad ska visas? Att algoritmen stannar, och att den svarar ja *om och endast om* en giltig placering finns. Båda riktningarna.

Lemma 1. En sammanhängande komponent har som mest två placeringar utan konflikt: BFS-färgningen och dess spegelbild. Ser BFS en kant inom en färg finns ingen.

Lemma 2. $M_i[s]$ är sant ⇔ komponent 1..i kan orienteras tillåtet med s katter i rum 1. Induktion över i.

Terminering: varje katt köas en gång, och alla slingor är begränsade.

Svarar ja ⇒ placering finns

Välj s i fönstret med $M_t[s]$. Lemma 2 ger tillåtna orienteringar: $s \le c_1$ och $n - s \le c_2$, alla trivs, och varje kant går mellan färgerna inom sin komponent.

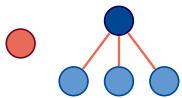
Placering finns ⇒ svarar ja

Grafen är bipartit. På varje komponent är placeringen en av de två orienteringarna (Lemma 1), och den är tillåten. Dess s ligger i fönstret, och $M_t[s]$ är sant (Lemma 2).

Varför inte girigt?

Girigt: ta komponenterna en i taget och lägg den större sidan i rummet med mest plats kvar.

Instansen



En ensam katt, och en stjärna med sidorna 1 och 3. Alla trivs överallt. $c_1 = 2$, $c_2 = 3$.

Girigt, ensam katt först





Tre löv, men bara två platser kvar i varje rum. ✗

Det finns en placering



Den ensamma katten och mitten i rum 1, löven i rum 2. ✓

Att ta komponenterna med flest katter först hjälper inte

Sidorna $(3, 1)$ och $(3, 2)$ med $c = (4, 5)$ fäller den varianten. Komponenterna hänger ihop via en summa, och det är därför det blir dynamisk programmering.

Vanliga fel i uppgift 2

- **”Det räcker att grafen är bipartit.”** Trivseln och kapaciteterna glöms.
- **En orientering per komponent.** Färg 0 hamnar alltid i rum 1 och spegelbilden prövas aldrig. Eller kontrolleras trivseln för en orientering, och sedan vänds komponenter fritt.
- **Girig orientering**, se föregående bild.
- **Tabell upp till** c_1 utan att kapa: $O(n \cdot c_1)$ är inte polynomiskt. Omvänt: ”delmängdssumma är NP-svårt, så det går inte”. Här är talen högst n.
- **Små fall som fäller koden:** en ensam katt; en katt med $R_i = \emptyset$; $<$ i stället för $\leq$ mot kapaciteten; en startkatt i BFS som aldrig markeras och därför räknas två gånger.
- **Korrekthet åt ett håll**, eller utan terminering.
- **För långt:** gränsen är 1 500 ord per uppgift, pseudokoden inräknad.

Testa på två katter

Nästan varje fel ovan syns redan med en eller två katter. Kör dem för hand innan du lämnar in.

Del 3 av 3

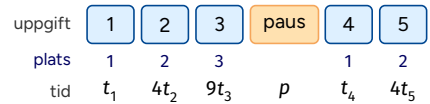
Fikaoptimering

dynamisk programmering under ett tak



Uppgiften

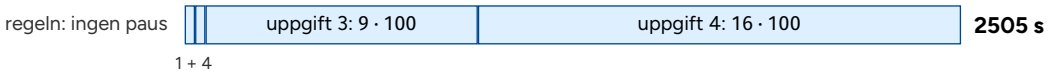
- n uppgifter i given ordning. Uppgift i tar t_i sekunder direkt efter en paus.
- Mellan två uppgifter kan vi ta en fikapaus på $p = 1800$ sekunder.
- Den j:te uppgiften sedan senaste paus tar $j^2 \cdot t_i$. Uppgift 1 görs direkt efter frukost och tar t_1.
- Hitta en **bästa placering av pauserna** – inte bara den minsta tiden.
- Så snabbt som möjligt, och visa att det är optimalt. Korrekthet med invarianter, och inga hjälpalgoritmer utan egen pseudokod.



Pausen nollställer räknaren j. Den kostar 1800 sekunder, men gör varje följande uppgift i blocket billigare.

Varför inte girigt?

Regeln ”pausa före uppgift i om $j^2 t_i > p + t_i$” jämför bara den aktuella uppgiften. Exempel: $t = (1, 1, 100, 100)$.

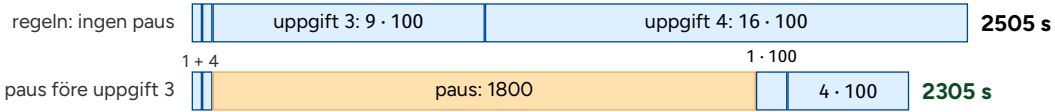


Kortsiktigt

900 < 1900 och 1600 < 1900, så regeln pausar aldrig. Men en paus före uppgift 3 gör *både* 3 och 4 billigare, och det ser inte regeln.

Varför inte girigt?

Regeln ”pausa före uppgift i om $j^2 t_i > p + t_i$” jämför bara den aktuella uppgiften. Exempel: $t = (1, 1, 100, 100)$.



Kortsiktigt

900 < 1900 och 1600 < 1900, så regeln pausar aldrig. Men en paus före uppgift 3 gör *både* 3 och 4 billigare, och det ser inte regeln.

Och det kan bli mycket värre

Med alla $t_i = 1$ och $n = 43$ pausar regeln först vid $j = 43$: 27 386 s. Optimum är 6 870 s, med block på 14–15 uppgifter.

Block längre än 42 lönar sig aldrig



Lägg en paus före den sista uppgiften i ett block med längd L. Tiden ändras med

$$(p + t) - L^2 t = p - (L^2 - 1)\,t \;\le\; 1800 - 1848 < 0$$

när $t \ge 1$ och $L \ge 43$. Inget annat ändras: efter blockets sista uppgift kommer ändå en paus, eller slutet.

Alltså har varje optimal plan block på högst $\lfloor \sqrt{p + 1} \rfloor = 42$ uppgifter.

"$\sqrt{1800} \approx 42$" är ingen motivering

Det är bytet som visar att ett längre block aldrig behövs.

Gäller för varje instans

Även för varje prefix $1..i$, som tabellen räknar på. Och 42 är bara en övre gräns.

Delproblemen: $B[i,j]$

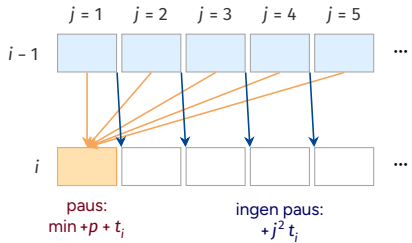
$B[i,j]$ = minsta tiden för uppgift 1..i, där uppgift i är den j**:te sedan senaste paus**. ($j = i$: ingen paus än.)

$$B[1,1] = t_1$$

$$B[i,1] = \min_{1 \le q \le \min(i-1,42)} B[i-1,q] + p + t_i$$

$$B[i,j] = B[i-1,\, j-1] + j^2\, t_i, \quad 2 \le j \le \min(i,42)$$

Svaret är $\min_j B[n,j]$. Tabellen har högst $42n$ celler och fylls rad för rad.

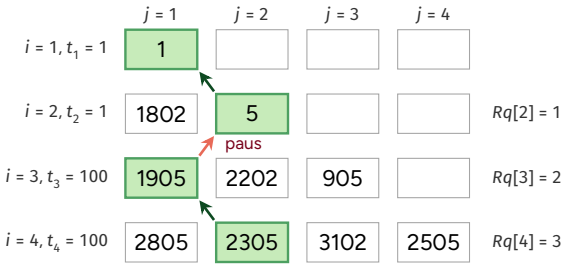


Exempel: $t = (1, 1, 100, 100)$

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	
$i = 1, t_1 = 1$	1				
$i = 2, t_2 = 1$	1802	5			$Rq[2] = 1$
$i = 3, t_3 = 100$	1905	2202	905		$Rq[3] = 2$
$i = 4, t_4 = 100$	2805	2305	3102	2505	$Rq[4] = 3$

Raderna fylls uppifrån. $B[3, 1] = \min(1802, 5) + 1800 + 100 = 1905$, med $Rq[3] = 2$.

Exempel: $t = (1, 1, 100, 100)$



Raderna fylls uppifrån. $B[3, 1] = \min(1802, 5) + 1800 + 100 = 1905$, med $Rq[3] = 2$.

Minimum på sista raden är $B[4, 2] = 2305$. Gå bakåt:

- $j = 2$: uppgift 3 kom direkt före, på plats 1.
- $j = 1$: paus före uppgift 3. Uppgift 2 hade plats $Rq[3] = 2$.
- $j = 2$: uppgift 1 på plats 1.

Planen: **1, 2, paus, 3, 4** – 2305 sekunder.

Pseudokod: tabellen

```
    FikaOpt(n, t[1..n]) =
1       p ← 1800;  q_max ← ⌊√(p + 1)⌋     // = 42
2       B[1, 1] ← t[1]
3       for i ← 2 to n do
4           inv B[k, ·] är korrekt för varje k < i
5           M ← B[i − 1, 1];  Rq[i] ← 1
6           for j ← 2 to min(i − 1, q_max) do
7               inv M = min_{1≤q<j} B[i − 1, q], uppnått för q = Rq[i]
8               if B[i − 1, j] < M then M ← B[i − 1, j]; Rq[i] ← j
9           B[i, 1] ← M + p + t[i]
10          for j ← 2 to min(i, q_max) do
11              inv B[i, 1..j − 1] är korrekt
12              B[i, j] ← B[i − 1, j − 1] + j² · t[i]
```

- **Rad 5–8** hittar minimum av förra raden och kommer ihåg var det låg: $Rq[i]$ behövs för att hitta tillbaka.
- **Rad 9** är en paus före uppgift i, rad 10–12 ingen paus.
- Varje varv av den yttre slingan gör högst $2 \cdot 42$ inre varv.

Pseudokod: svaret och planen

```
13    j ← 1
14    for k ← 2 to min(n, q_max) do
15        inv B[n, j] = min_{1≤q<k} B[n, q]
16        if B[n, k] < B[n, j] then j ← k
17    plan ← tom lista;  u ← n
18    while u > 1 do
19        inv plan är uppgift u+1..n med pauser, uppgift u
              har plats j, och B[u, j] + tiden för plan = OPT
20        lägg u först i plan
21        if j = 1 then lägg ”paus” först i plan;  j ← Rq[u]
22        else j ← j − 1
23        u ← u − 1
24    lägg 1 först i plan
25    return plan
```

Varför planen blir rätt

Varje steg bakåt följer den term som gav minimum:
$B[u, j] = B[u-1, j-1] + j^2 t_u$ när $j > 1$, och
$B[u, 1] = B[u-1, Rq[u]] + p + t_u$.
Invarianten håller i varje varv.
När $u = 1$ är $j = 1$ och $B[1, 1] = t_1$, så planen tar OPT.

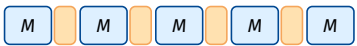
Lösningsförslaget slutar mitt i meningen ”Andra delen av algoritmen konstruerar en korrekt lösning eftersom …”. Rutan ovan är ett sätt att avsluta den.

Tidskomplexitet och undre gräns

Tid: $O(42\,n) = O(n)$ för tabellen och $O(n)$ för planen.

Undre gräns, $\Omega(n)$**:** ett motståndarargument. Låt $M > 600$.

$t = (M, M, M, M, M)$: paus före varje uppgift



$t = (M, M, 1, M, M)$: ingen paus före uppgift 3



Plats 2 kostar $4M > p + M$, så varje optimum pausar före varje uppgift. Med $t_k = 1$ kostar plats 2 bara $4 < p + 1$. En algoritm som aldrig läser t_k ger samma plan för båda, och har fel på den ena. Alltså måste $t_2, \ldots, t_n$ läsas.

Eller: utdatas längd

Med alla $t_i = M$ innehåller den enda optimala planen $n - 1$ pauser, och alla måste skrivas ut.

”Man måste läsa all indata”

räcker inte: man måste säga varför varje t_k spelar roll. t_1 behöver faktiskt aldrig läsas för planen – den förskjuter varje plan lika mycket.

Korrekthet för dynamisk programmering

Vad som allmänt ska visas

1. Rekursionen och basfallen är rätt: optimal delstruktur.
2. Beräkningsordningen räknar ut varje värde innan det används.
3. Svaret läses ur rätt celler. Här är det lemmat om 42 som gör att kolumnerna räcker.
4. Rekonstruktionen ger en plan med det värdet.
5. Alla slingor terminerar.

Rekursionen i uppgift 3

$j > 1$**:** uppgift i följer direkt på $i - 1$, som då har plats $j - 1$. Det som hände före påverkar inte vad uppgift i kostar, så resten måste vara optimalt för $B[i - 1, j - 1]$.

$j = 1$**:** en paus före i. Uppgift $i - 1$ har någon plats $q \leq 42$ (lemmat), och den bästa väljs.

Kravet för A

Invarianter för alla slingor, även den yttersta. De får komma muntligt.

Vanliga fel i uppgift 3

- **Bara minimivärdet.** Uppgiften kräver pausernas placering. Ett tal räcker inte, hur rätt det än är.
- **Den giriga regeln** ”pausa när $j^2 t_i > p + t_i$”.
- **”42 eftersom $\sqrt{1800} \approx 42$”** utan bytesargumentet.
- $O(n^2)$ **utan tak.** Polynomiskt räcker för C, men A kräver linjär tid.
- **Undre gränsen** som ”man måste läsa indata” utan skäl till varför varje t_k spelar roll, eller som en räkning av slingvarv. Fel eller saknad skriftligt ger ett mindre fel som inte går att reparera muntligt.
- **Invarianter** som egentligen är eftervillkor, fel indexintervall, eller ingen invariant för den yttre slingan.
- **Kanterna:** en paus före uppgift 1 eller efter den sista, eller en markör som hamnar i utdata.

Formalia som kostade

- **Redogörelsen för externa resurser och AI** krävs även när inget använts. Saknas den när redovisningen börjar underkänns uppgifterna.
- **Copilot** kräver en bilaga med chattloggarna.
- **Högst 1 500 ord** per uppgift, pseudokoden inräknad.
- **Bilden** i uppgift 1 är ett krav, inte en dekoration.

Hjälpalgoritmer

I uppgift 2 får kursbokens algoritmer, som BFS, användas med en hänvisning och sidnummer. I uppgift 3 ska varje hjälpalgoritm finnas med i pseudokoden.

Och vid redovisningen

Det är du som ska övertyga. Förbered en kort genomgång av varje lösning, och räkna ett litet exempel för hand.

Innehåll, som avslutning

1. **Sum Ergo Cogito** – dela talet på mitten och kvadrera fram 10^m: $O(n^{1,5})$.
2. **Hem för hemlösa katter** – högst två orienteringar per komponent, och en tabell över antalet katter i rum 1: $O(n^2)$.
3. **Fikaoptimering** – block på högst 42, tabellen $B[i, j]$ och planen bakåt: $O(n)$, och $\Omega(n)$ med ett motståndarargument.
4. **Formalia** – redogörelsen för resurser och AI, ordgränsen och bilden.

Felen i en mening

Konstanter som växer med n, och svar utan plan.



Frågor? Lösningsförslaget finns i Canvas.